

C++第五章

1.STL-Part2

- STL 容器总览:

- 定义: STL 容器是**同一类型**元素的集合。

常用容器一览: `vector`, `array`, `list`, `set`, `queue`, `stack`, `map`。

说明: 每种容器在特定场景很擅长, 在其他场景可能很差。

- 提出三个引导问题, 帮助思考“如何选容器”:

1. 需要**频繁插入/删除**用什么容器?
2. 需要**频繁合并/拆分** (例: 撮合系统的订单簿) 用什么容器?
3. 需要**随机访问**用什么容器?

- 默认首选 `vector`; 除非明显低效再考虑别的。
- 若元素**频繁插入/删除**: 用 `list` (但它**不支持按下标随机访问**, 需要随机访问就别用)。
- 需要**按键访问**: 用 `map`。
- 按算法需要 **FIFO/LIFO**: 用 `queue` / `stack`。
- 元素需要**互异且按某种性质去重/有序**: 用 `set`。

- STL `set` 基本用法

- 头文件: `#include <set>`。
 - **键唯一且有序** (默认按 `<` 升序)。
 - 常用操作与复杂度: `insert/find/erase` 均 $O(\log n)$; `count(k)` 在 `set` 中只有 0 或 1。
 - 迭代器是只读键 (修改会破坏有序性, 需“删后再插”)。
- 删除: `s.erase(e)` 按值删除。
- 查找: `s.find(e)` 找到则返回迭代器, 否则返回 `s.end()`; `s.count(e)` 在 `set` 中只会是 0 或 1。
- 示例:

```
1 set<string> visitors;
2
3 visitors.insert("Alice");
4 visitors.insert("Bob");
5 visitors.insert("Charlie");
6 visitors.erase("Alice");
7 cout << visitors.count("Alice") << '\n'; // 0 - false - not
8 found
9 cout << visitors.count("Charlie") << '\n'; // 1 - true - found
10 cout << (visitors.find("Zyler") == visitors.end()) << '\n'; // 1 -
    true - not found
```

- `set<int>` 示例:

```
1 #include <iostream>
```

```

2  #include <set>
3  using namespace std;
4
5  int main() {
6      set<int> numbers;
7      for (int i = 1; i <= 10; ++i) numbers.insert(i);
8
9      numbers.erase(5);
10
11     for (const auto& x : numbers) cout << x << ' '; // 1 2 3 4 6 7 8
12     cout << '\n';
13
14     auto it = numbers.find(5);
15     if (it != numbers.end()) cout << "Found\n";
16     else cout << "Not found\n"; // 这里将输出
17     Not found
18 }

```

- `std::map`：概念与基本操作

- 头文件：`#include <map>`
- 特性：**键有序、唯一**；节点键为 `const`（不能原地修改键）。
- 复杂度：`find/insert/erase` 均 $O(\log n)$ 。
- 常用接口与注意点：
 - `operator[]`：若键不存在会**插入默认值**并返回引用（读操作也会写入，谨慎使用）。
 - `at(key)`：仅访问，若不存在抛异常（不会插入）。
 - `insert({key, value})` / `emplace(key, value)`：插入并返回 `{iterator, bool}`。
 - `find(key)`：存在则返回迭代器，否则 `end()`。

示例：

```

1  #include <iostream>
2  #include <map>
3  using namespace std;
4
5  int main() {
6      map<char, int> my_map;
7      my_map['a'] = 0; // 不存在则插入 { 'a', 0 }
8      my_map['b'] = 3;
9      my_map['c'] = 40;
10
11     // 通过初始化列表构造另一个 map，并插入新键
12     map<char, int> second_map{{'a', 3}, {'b', 4}};
13     auto [it, inserted] = second_map.insert({'c', 40}); // inserted
14     = true
15
16     // 展示 "读时建键" 的副作用
17     cout << "my_map['x']=" << my_map['x'] << '\n'; // 输出 0，并
18     插入 {'x', 0}
19
20     // 只检查是否存在: find / at
21     auto f = my_map.find('y');

```

```

20     cout << (f == my_map.end() ? "y not found" : "y found") << '\n';
21
22     // 有序遍历
23     for (const auto& [k,v] : my_map) cout << '(' << k << ':' << v <<
24         ") ";
25     cout << '\n';

```

- 在 `map` 中查找元素 (To find an element in map)

要点

- `std::map::find(k)`：若存在返回指向该键值对的迭代器，否则返回 `end()`。
- 访问值：用 `it->second`。
- 复杂度： $O(\log n)$ (平衡树)。
- 不要用 `operator[]` 来“试着读”，因为会在不存在时插入默认值 (读时建键副作用)。

示例

```

1  #include <map>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      map<char,int> my_map{{'a',10}, {'b',20}, {'c',40}};
7
8      if (auto it = my_map.find('c'); it != my_map.end()) {
9          cout << it->second << '\n'; // 40
10     } else {
11         cout << "not found\n";
12     }
13 }

```

- 遍历 `map` (Iterate over a map)

三种常见方式

1. C++17 结构化绑定 (只读)

```

1  for (const auto& [key, val] : my_map) {
2      cout << key << " => " << val << '\n';
3  }

```

- `key` 实际类型是 `const Key` (键在 `map` 中为只读)。

1. C++17 结构化绑定 (修改 value)

```

1  for (auto& [key, val] : my_map) { // 注意使用 & 引用
2      // key 仍是 const, 不能改; 可以修改 val
3      ++val;
4  }

```

1. C++17 之前 (显式迭代器)

```

1 for (auto it = my_map.begin(); it != my_map.end(); ++it) {
2     cout << it->first << " => " << it->second << '\n';
3 }

```

复杂度：遍历是线性的 $O(n)$ ；在遍历中修改 **value** 安全，修改 **key** 不被允许（键是 `const`），若需改变“排序关键字”应“删旧键再插入新键”。

- 以结构体作为 `map` 的键（STL map with struct）

核心点

- map 的“键等价性”由比较器决定（默认 `std::less<Key>`，即 `operator<`）。
- 若把 `struct` 作为键，必须提供**严格弱序**（strict weak ordering）的比较：
 - 在 `struct` 内定义 `operator<`，或
 - 在 `map` 模板参数中提供**自定义比较器类型**（函数对象 / lambda / `std::function` 等）。
- 若比较器把两个不同对象判为“等价”（两边都不小于对方），则它们**不能共存为两个键**。如果需要“允许相等键并存”，应使用 `std::multimap`。

方式 A：在结构体内定义 `operator<`（推荐、零开销）

```

1 #include <map>
2 #include <string>
3 #include <tuple>
4 using namespace std;
5
6 struct Student {
7     string name;
8     int score;
9     // 定义严格弱序：按 score 降序，再按 name 升序打破并列
10    bool operator<(const Student& other) const {
11        if (score != other.score) return score > other.score;
12        // 降序
13        return name < other.name;
14        // 次关键字
15    }
16 };
17
18 int main() {
19     map<Student, string> students; // 比较使用 Student::operator<
20     students.insert({"Bob", 90}, "B+");
21     students.insert({"Bob", 90}, "B+"); // 等价键（name+score 相同）不会重复插入
22 }

```

方式 B：提供比较器类型 / lambda（需要把对象传入构造函数）

```

1 #include <map>
2 #include <functional> // 仅当使用 std::function 时需要
3
4 struct Student { std::string name; int score; };
5
6 int main() {
7     // 1) 非捕获 lambda，作为比较器对象
8 }

```

```

8     auto cmp = [](const Student& a, const Student& b) {
9         if (a.score != b.score) return a.score > b.score;
10        // 降序
11        return a.name < b.name;
12        // 次关键字
13    };
14    // 2) 使用 decltype(cmp) 作为模板参数 (零开销)
15    std::map<Student, std::string, decltype(cmp)> m(cmp);
16    // 3) 或者使用 std::function<bool(const Student&, const
17    Student&>
18    // 但有类型擦除开销, 不如 2) 高效:
19    // std::map<Student, std::string,
20    // std::function<bool(const Student&, const Student&>>
21    m2(cmp);
22    m.insert({"Bob", 90}, "B+");
23    m.insert({"Alice", 95}, "A");
24 }

```

- STL 算法概览 (STL algorithms)

为什么 `map/set` 有成员 `find`, 但 `vector/list` 没有?

- `map/set` 的查找依赖其内部有序/哈希结构, 提供成员 `find` 能利用结构实现 $O(\log n)$ (或 $O(1)$ 哈希)。
- `vector/list` 则是序列容器, 通用查找由 `<algorithm>` 的泛型算法完成: `std::find`、`std::count` 等。

算法来自哪些头文件?

- 主要在 `<algorithm>`; 数值类算法在 `<numeric>` (如 `accumulate`、`inner_product` 等)。

使用 STL 算法的三大优势

1. 与容器解耦: 对迭代器而非容器编程, 适用于多种容器;
2. 高效: 标准库实现通常经过良好优化;
3. 易用: 表达意图清晰 (如 `std::sort`、`std::transform`)。

常见类别 (原页列出的四类, 并补充示例)

1. **Searching:** `find` / `binary_search` (要求已排序) / `lower_bound` / `upper_bound`
2. **Sorting:** `sort` (随机访问迭代器, 如 `vector/array/deque`), `list` 使用成员函数 `list::sort`
3. **Copying/Moving:** `copy` / `copy_if` / `move` / `unique_copy`
4. **Applying (对区间施加函数):** `for_each` / `transform` / `accumulate` (在 `<numeric>`)

示例: 把偶数提取并平方到目标容器

```

1 #include <vector>
2 #include <algorithm>
3 #include <numeric>
4 using namespace std;
5
6 int main() {

```



```

7     vector<int> v{1,2,3,4,5,6}, out;
8     out.reserve(v.size());
9
10    // copy_if + transform 的一种管道式写法 (两步)
11    vector<int> evens;
12    copy_if(v.begin(), v.end(), back_inserter(evens), [](int x){ return
13    x%2==0; });
13    transform(evens.begin(), evens.end(), back_inserter(out), [](int x){
14    return x*x; });
14 }

```

- Algorithm and Iterator

- STL 算法通常需要迭代器作为参数来指定范围或位置。

- 常见用法:

- `x.begin()` 和 `x.end()` 表示整个范围。

- `x.rbegin()` 和 `x.rend()` 可以反向操作。

- 子区间使用 `next` 和 `prev`:

- `next(X.begin(), 3), next(X.begin(), 5)` → 第 3 到第 4 个元素。

- `next(X.begin(), 1), prev(X.end(), 1)` → 第 2 到倒数第 2 个元素。

- 注意事项:

- 如果误把两个不同容器的迭代器混用 (如 `sort(X.begin(), Y.end())`) , 会导致运行时错误。

- 如果传入非法区间 (如 `x.end(), x.begin()`) , 可能会产生无限循环。

- Algorithm and Lambda Function

- 许多算法需要传入一个函数参数, 用来指定操作。

- 传入的函数可以是普通函数, 更多时候是 **lambda 表达式**。

- 输入通常是容器中的元素, 返回值可以是 `void`、`bool` 或一个值。

- STL Algorithms - Partial List

- **填充类 (fill)** : `fill`, `iota`, `generate`

- **归约类 (reduction)** : `all_of`, `any_of`, `none_of`, `min_element`, `max_element`, `accumulate`, `reduce`, `reverse`

- **遍历类 (for)** : `for_each`, `transform`

- **条件类 (if)** : `copy`, `copy_if`, `count`, `count_if`, `find`, `find_if`, `find_if_not`

- **排序与搜索类 (sort)** : `sort`, `binary_search`, `search`

- 示例: 填入一个值 fill a value

```

1  #include <algorithm>
2  using std::fill;
3
4  array<int, 3> arr;
5  fill(arr.begin(), arr.end(), 3);

```

- **iota**

- `iota` 来自 `<numeric>`, 用于将容器按递增序列填充。

- 使用方式:

- `iota(seq.begin(), seq.end(), 0);` → 从 0 开始依次填充。
- `iota(seq.rbegin(), seq.rend(), 0);` → 反向填充。

示例:

```
1 // 1.15 iota
2 #include <numeric>
3 #include <array>
4 using namespace std;
5
6 array<int, 7> seq;
7 iota(seq.begin(), seq.end(), 0); // 0,1,2,3,4,5,6
8 iota(seq.rbegin(), seq.rend(), 0); // 反向填充
```

- **generate**

- `generate` 可以使用一个函数 (或 lambda) 生成值来填充容器。
- 例子 1: 用计数器生成依次递增的平方数。
- 例子 2: 用随机数分布生成随机数。

示例:

```
1 // 1.16 generate
2 #include <vector>
3 #include <random>
4 #include <cmath>
5 using namespace std;
6
7 vector<int> u(20);
8 int cc = 0;
9 generate(u.begin(), u.end(), [&cc]() { cc++; return cc * cc; });
10
11 // 随机数生成器
12 mt19937_64 mt{100};
13 student_t_distribution<> tdist(5); // 自由度 5
14 generate(u.begin(), u.end(), [&mt, &tdist]() { return tdist(mt); });
```

- **all_of / any_of / none_of**

- 接收一个谓词函数 (返回布尔值), 并对整个区间进行判断:
 - `all_of`: 所有元素满足条件返回 true。
 - `any_of`: 至少一个元素满足条件返回 true。
 - `none_of`: 没有任何元素满足条件返回 true。

例子:

- 判断是否全是偶数。
- 判断是否全为正数。

```

1 // 1.17 all_of / any_of / none_of
2 #include <algorithm>
3 #include <iostream>
4 using namespace std;
5
6 vector<int> v{1, 2, 3, 4, 5};
7 bool all_even = all_of(v.begin(), v.end(), [](int i){ return i % 2 == 0;
8 });
9 bool all_positive = all_of(v.begin(), v.end(), [](int i){ return i > 0;
10 });

```

- **max_element / min_element / accumulate / reduce / reverse**

- **max_element**：找到最大值元素的迭代器。
- **min_element**：找到最小值元素的迭代器。
- **accumulate**：对序列做累计求和（或其他操作）。
- **reduce**：C++17 新增，和 **accumulate** 类似，但可并行。
- **reverse**：原地反转容器。

示例：

```

1 // 1.18 max_element / min_element / accumulate / reduce / reverse
2 #include <numeric>
3 vector<int> seq2{0, 1, 2, 3, 17, 18, 19};
4
5 auto it_max = max_element(seq2.begin(), seq2.end());
6 auto it_min = min_element(seq2.begin(), seq2.end());
7
8 cout << *it_max << "\n";
9 cout << *it_min << "\n";
10
11 double avg1 = accumulate(v.begin(), v.end(), 0.0) / v.size();
12 double avg2 = reduce(v.begin(), v.end(), 0.0) / v.size(); // C++17
13
14 reverse(v.begin(), v.end()); // 原地反转

```

- **for_each**

- **for_each** 是泛型且灵活的算法，它接受一个函数，该函数从容器中获取值作为输入。
- 可以替代 **for** 循环来遍历容器。
- 当输入是引用类型时，可以修改容器中的值。
- **限制**：
 - 不能使用 **break** 或 **continue**。
 - 没有迭代器变量来追踪位置。

示例：

- 打印输出。
- 使用引用修改元素值。


```

1 // 1.19 for_each 基本用法
2 vector<int> v{1, 2, 3, 4, 5};
3
4 // 输出元素
5 for_each(v.begin(), v.end(), [](int x){ cout << x << ", "; });
6 cout << '\n';
7
8 // 修改元素
9 for_each(v.begin(), v.end(), [](int &x){ x = x * 2; });
10 cout << '\n';

```

- **for_each - more examples**

- **错误示例:** `for_each(v.end(), v.begin(), ...)` 会导致无限循环。
- **增量赋值:** 可用 `iota` 替代。
- **求和:** 可用 `accumulate` 替代。
- **计数偶数:** 可用 `count_if` 替代。

```

1 // 1.20 for_each 更多示例
2 // 错误: 会无限循环
3 // for_each(v.end(), v.begin(), [](int x){ cout << x << " "; });
4
5 int cc = 11;
6 for_each(v.begin(), v.end(), [&cc](int &x){ x = cc++; });
7 cout << '\n';
8
9 int sum = 0;
10 for_each(v.begin(), v.end(), [&sum](const int &x){ sum += x; });
11 cout << sum << '\n';
12
13 int count_even = 0;
14 for_each(v.begin(), v.end(), [&count_even](const int &x){ count_even +=
15 (x % 2 == 0); });
16 cout << count_even << '\n';

```

- **transform**

- `transform` 接收一个函数，将容器的值作为输入，并返回一个值写入容器。
- 可以用来修改容器，也能用一个或两个容器的元素来生成另一个容器。
- 常配合 `back_inserter` 使用，自动扩展目标容器。
- 如果不使用 `back_inserter`，需要确保目标容器有足够空间，否则可能运行时错误。
- **常见变体:**
 1. 输入和输出是同一个容器（更新自身）。
 2. 更新另一个容器。
 3. 用两个容器生成另一个容器。
 4. 用两个容器更新自身。

```

1 // 1.21 transform
2 vector<int> out(v.size());
3
4 // 1. 更新自身
5 transform(v.begin(), v.end(), v.begin(), [](int x){ return x * x; });
6
7 // 2. 更新另一个容器
8 transform(v.begin(), v.end(), out.begin(), [](int x){ return x + 10; });
9
10 // 3. 两个容器生成另一个容器
11 vector<int> v2{10, 20, 30, 40, 50};
12 vector<int> out2(v.size());
13 transform(v.begin(), v.end(), v2.begin(), out2.begin(), [](int a, int b)
14 { return a + b; });

```

- transform — illustration (4 种形态)

一元，就地更新：输入与输出是同一容器，形如

$$v[i] \leftarrow f(v[i])$$

一元，写到另一容器：

$$\text{out}[i] \leftarrow f(v[i])$$

二元，就地更新：同一容器既作输入又作输出（少见，需保证语义正确）：

$$v[i] \leftarrow g(v[i], w[i])$$

二元，写到另一容器：常用于逐元素合并两个容器：

$$\text{out}[i] \leftarrow h(v[i], w[i])$$

transform — example

- **就地更新：**把每个元素变成原来的两倍。
- **写到新容器（两种做法）：**
 - 预先分配同尺寸的目标容器；
 - 用 `back_inserter` 让输出容器自动扩展（需 `#include <iterator>`）。

```

1 // 1.23 就地更新: x <- 2*x
2 vector<int> v{1,2,3,4,5};
3 transform(v.begin(), v.end(), v.begin(),
4           [](int x){ return x * 2; });
5
6 // 1.23 写到新容器: 方式一——预先分配同尺寸
7 vector<int> v2(v.size());
8 transform(v.begin(), v.end(), v2.begin(),
9           [](int x){ return x * 2; });
10
11 // 1.23 写到新容器: 方式二——back_inserter 自动扩展
12 vector<int> v3;
13 transform(v.begin(), v.end(), back_inserter(v3),
14           [](int x){ return x * 2; });

```

- transform — more examples -1

- “**v 的两倍**”：把 `v` 同自己逐元素相加，得到 $v + v$ 。
- “**v + v2 = v3**”：二元版本把两个容器逐元素相加到第三个容器。

```

1 // 1.24 示例一：把 v 与自身相加，得到 “v 的两倍”
2 vector<int> v_twice(v.size());
3 transform(v.begin(), v.end(), // 输入 1
4           v.begin(),           // 输入 2 (同一个 v)
5           v_twice.begin(),     // 输出
6           [](int a, int b){ return a + b; });

8 // 1.24 示例二：v + w -> out
9 vector<int> w{10,20,30,40,50};
10 vector<int> out(v.size());
11 transform(v.begin(), v.end(),
12           w.begin(),
13           out.begin(),
14           [](int a, int b){ return a + b; });

```

- transform — more examples - 2 (时间序列收益率)
 - 给定按时间升序的价格序列 $p_0, p_1, \dots, p_n$ ，计算相邻收益率

$$r_t = \frac{p_t}{p_{t-1}} - 1$$

做法：用 `next(v1.begin())` 与 `v1.begin()` 作为两路输入，输出长度为 `size()-1`。

```

1 // 1.25 时间序列收益率 r_t = p_t / p_{t-1} - 1
2 vector<double> price{1.0978, 1.0959, 1.1003, 1.0952, 1.1008};
3 vector<double> ret(price.size() - 1);
4 transform(next(price.begin()), price.end(), // 输入 1: p_t
5           price.begin(),                   // 输入 2: p_{t-1}
6           ret.begin(),
7           [](double pt, double ptm1){ return pt / ptm1 - 1.0; });

8
9 // (可选) 打印结果检验
10 for (double r : ret) cout << r << " ";
11 cout << "\n";

```

- **copy / copy_if**
 - `copy`：把一个容器的数据复制到另一个容器。
 - `copy_if`：只复制满足条件的元素。
 - 如果目标容器大小未知，可以用 `back_inserter` 扩展容器，但性能比预先分配差。

示例：

```

1 // 1.26 copy
2 list<int> L1{1,2,3,4,5};
3
4 // 方法一：目标容器预分配好大小
5 vector<int> xs(L1.size());
6 copy(L1.begin(), L1.end(), xs.begin());
7
8 // 方法二：back_inserter 自动扩展
9 vector<int> xs2;
10 copy(L1.begin(), L1.end(), back_inserter(xs2));

```

- **copy helps conversion between containers**

- copy 可以用于不同类型容器之间的转换（例如 `vector<int>` → `vector<double>`，或 `list` → `vector`）。
- 注意：目标容器必须有足够的空间。

示例：

```

1 // 1.27 copy 转换容器
2 auto copy_int_to_double = [](){
3     vector<int> ivector(100, 1);
4     vector<double> dvector(100);
5     copy(ivector.begin(), ivector.end(), dvector.begin());
6     return dvector;
7 };
8
9 void from_list_to_vector(const list<int>& L){
10     vector<int> values(L.size());
11     copy(L.begin(), L.end(), values.begin());
12 }

```

- **copy to itself**

- 如果在同一个容器内拷贝，要避免“源区间”和“目标区间”重叠。
- **正确方法**：先 `resize` 容器，再用 `copy`。
- **错误方法**：边 `resize` 边用 `back_inserter`，可能因迭代器失效导致错误数据。

```

1 // 1.28 copy to itself
2 vector<int> v1{1,2,3,4,5};
3 // 正确：resize 再 copy
4 v1.resize(v1.size() + 3);
5 copy(v1.begin(), next(v1.begin(), 3), next(v1.begin(), 5));
6 // 结果：1,2,3,4,5,1,2,3
7
8 // 错误：同时 resize + back_inserter
9 vector<int> v2{1,2,3,4,5};
10 copy(v2.begin(), next(v2.begin(), 3), back_inserter(v2));
11 // 结果不确定，可能出错

```

- **copy_if**

- `copy_if` 结果大小不确定，通常配合 `back_inserter` 使用。
- 可用于筛选符合条件的数据（例如 `opt.daysToExpiration < 10` 的对象）。

```

1 // 1.29 copy_if
2 vector<int> xs3;
3 copy_if(L1.begin(), L1.end(), back_inserter(xs3),
4         [](auto x){ return x > 3; });
5 // xs3 = {4, 5}
6
7 // 结构体筛选
8 struct StandardOption {
9     double strike;
10    int daysToExpiration;
11 };
12 vector<StandardOption> opts = {
13     {100.5, 5}, {101.2, 15}, {99.9, 7}
14 };
15 vector<StandardOption> result;
16 copy_if(opts.begin(), opts.end(), back_inserter(result),
17         [](auto opt){ return opt.daysToExpiration < 10; });
18 // result 包含 daysToExpiration 小于 10 的元素

```

- copy to output
 - 可以把容器内容直接“复制”到 `cout`。
 - 常用 `ostream_iterator<T>` 作为输出迭代器；当然，普通 `for` 循环更直观。

```

1 // 1.30 copy to output
2 vector<double> prices{1.2, 2.3, 3.4};
3 for (auto v : prices) { cout << v << ", "; }
4 cout << '\n';
5
6 copy(prices.begin(), prices.end(),
7      ostream_iterator<double>(cout, ", "));
8 cout << '\n';

```

- count / count_if
 - `count(first, last, value)`：统计等于某个值的个数。
 - `count_if(first, last, pred)`：统计满足谓词（返回 `bool`）的个数。
 - 示例含：寻找等于3的个数、统计大于3的个数（`lambda`与`bind`两种写法）。

```

1 // 1.31 count / count_if
2 vector<int> v{1, 2, 3, 4, 5, 3, 2, 1};
3 int three = 3;
4 auto num_threes = count(v.begin(), v.end(), three); // 等于 3 的个数
5
6 // 大于 3 的个数 (lambda)
7 auto is_above_3 = [](int x){ return x > 3; };
8 auto num_above_3_1 = count_if(v.begin(), v.end(), is_above_3);
9
10 // 同样目的 (直接内联 lambda)
11 auto num_above_3_2 = count_if(v.begin(), v.end(), [](int x){ return x > 3; });
12
13 // 使用二元谓词 + bind: greater(a,b) => a > b
14 auto greater2 = [](int a, int b){ return a > b; };

```



```

15 auto num_above_3_3 = count_if(v.begin(), v.end(),
16                               bind(greater2, placeholders::_1, 3));

```

bind 的作用

```

1 bind(greater2, placeholders::_1, 3)

```

- `std::bind` 可以把一个多参函数“绑定”成少参函数。
- `placeholders::_1` 表示未来调用时传进来的第一个参数。
- `3` 是固定值。

所以这句绑定成了一个 **一元函数**：

```

1 f(x) = greater2(x, 3) // 等价于 return x > 3

```

在 `count_if` 中的应用

```

1 auto num_above_3_3 = count_if(v.begin(), v.end(),
2                               bind(greater2, placeholders::_1, 3));

```

- `count_if` 会对容器 `v` 中的每个元素调用谓词函数。
- 谓词是 `f(x) = x > 3`。
- 结果：统计 `v` 中 **大于 3** 的元素个数。

和直接 `lambda` 的对比

其实这段代码等价于更简单的写法：

```

1 auto num_above_3 = count_if(v.begin(), v.end(),
2                             [](int x){ return x > 3; });

```

使用 `bind` 的意义在于：

- 可以把已有的二元函数改造为一元函数，方便复用。
- 更接近函数式编程风格。
- `find` / `find_if` / `find_if_not`
 - 都**返回迭代器**。
 - `find(pos1, pos2, e)`：等于 `e` 的第一个位置；若返回值 $\neq$ `pos2`，则找到了。
 - `find_if(pos1, pos2, pred)`：第一个使谓词为真的位置。
 - `find_if_not(pos1, pos2, pred)`：第一个使谓词为假的位置。
 - 想要“找出所有匹配项”，需循环调用或用 `copy_if` 复制出来。

Example for `find`

- 在 `vector<int>` 中找出**所有等于 3 的元素**：
 - 反复从上一次位置的下一个迭代器继续 `find`；
 - 将迭代器转成**索引**保存（用 `distance` 计算）。

示例：

```

1 // 1.32 / 1.33 find 的用法: 找出所有等于 3 的位置并打印
2 vector<int> idx; // 保存索引
3 auto it = v.begin();
4 while (it != v.end()) {
5     it = find(it, v.end(), 3);
6     if (it == v.end()) break;
7     cout << *it << '\n'; // 打印找到的值 (都是 3)
8     idx.push_back(distance(v.begin(), it)); // 把当前找到的元素在容器中的位置存
    到 idx
9     ++it; // 移到下一个位置
10 }
11 cout << "loc: ";
12 for (auto k : idx) cout << k << ", ";
13 cout << '\n';

```

• sort

- `sort(pos1, pos2)`: 默认使用 `<` 运算符, 按升序排序。
- 也可以传入自定义比较函数 (lambda)。
- 可以跟踪比较次数, 验证排序复杂度。

示例:

```

1 // 1.34 sort
2 vector<int> v{2, 3, 1};
3 sort(v.begin(), v.end()); // 默认升序
4
5 // 降序排序
6 sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
7
8 // 计数比较次数
9 size_t comp_counter = 0;
10 vector<int> vec{3, 1, 4, 2};
11 sort(vec.begin(), vec.end(),
12     [&comp_counter](int a, int b){
13         ++comp_counter;
14         return a > b;
15     });

```

• sort for struct

- 结构体排序时, 需要指定排序字段。
- 可用 lambda 函数, 例如根据成员 `strike` 排序。

```

1 // 1.35 sort for struct
2 struct Option {
3     double strike;
4 };
5 vector<Option> vopt{{110.0}, {100.0}, {95.0}};
6 sort(vopt.begin(), vopt.end(),
7     [](const Option &a, const Option &b){
8         return a.strike < b.strike;
9     });
10 cout << vopt.front().strike << '\n'; // 95
11 cout << vopt.back().strike << '\n'; // 110

```

- **binary_search**

- `find` 是线性查找，复杂度 $O(n)$ 。
- `binary_search` 更快 ($O(\log n)$)，但 **只适用于已排序容器**。
- 示例：
 - 在未排序容器中，结果不正确。
 - 排序后使用 `binary_search`，结果正确。

```
1 // 1.36 binary_search
2 vector<int> v1{1, 3, 2, 5, 4};
3 cout << "unsorted\n";
4 for (auto val : array<int,5>{1,2,3,4,5})
5     cout << val << ": " << binary_search(v1.begin(), v1.end(), val) <<
6     '\n';
7 cout << "sorted\n";
8 sort(v1.begin(), v1.end());
9 for (auto val : array<int,5>{1,2,3,4,5})
10     cout << val << ": " << binary_search(v1.begin(), v1.end(), val) <<
11     '\n';
```

- **search for a sequence**

- `search(pos1, pos2, pos3, pos4)`：在 `[pos1, pos2]` 中查找子序列 `[pos3, pos4]`。
- 返回第一个匹配子序列的起始迭代器，若找不到则返回 `pos2`。

```
1 // 1.37 search for a sequence
2 vector<int> v2{1,2,3,4,5,3,2,1};
3 vector<int> u{3,4,5};
4 auto it = search(v2.begin(), v2.end(), u.begin(), u.end());
5 if (it != v2.end()) {
6     cout << *it << '\n';
7     cout << distance(v2.begin(), it) << '\n';
8 } else {
9     cout << "not found\n";
10 }
```

条件判断：如果 `it` 不是 `v2.end()`，说明找到了。

`*it`：解引用迭代器，得到“起点元素”的值，这里是 3。

`distance(v2.begin(), it)`：计算从起点 `v2.begin()` 到 `it` 的**距离**，在随机访问容器（如 `vector`）中就是**下标**。这里得到 2。

找不到时输出 `"not found"`。

- **Summary - STL algorithm**

- 自 1994 年以来，STL 算法已成为现代 C++ 的核心工具。
- 在金融编程中，STL 算法可操作数据序列，如曲线、时间序列、二维表面等。
- STL 算法优点：
 1. 泛型：可作用于任意容器。
 2. 高效：常可替代显式循环。

3. 灵活：配合 lambda 函数方便指定行为。

2.Input and Output

- **输入**：从文件或键盘获取数据。

输出：把结果输出到屏幕或写入文件。

主题包括：

1. I/O manipulators（输入输出操纵符，如 `std::setw`、`std::setprecision`）。
2. 文件 I/O（纯文本）。
3. 文件 I/O（二进制）。

- C++ I/O: overview

- 当程序生成大量数据时，建议写入文件。
- 在 C++ 中，通过“文件流”完成：
 - `ofstream` 写文件。
 - `ifstream` 读文件。
- `cout` → 屏幕，`stringstream` → 字符串，`fstream` → 文件，它们接口相同，非常方便。

表格总结：

Library	Stream	Device
iostream	<code>cin</code> / <code>cout</code>	screen
sstream	<code>istringstream</code> / <code>ostringstream</code> / <code>stringstream</code>	string
fstream	<code>ifstream</code> / <code>ofstream</code>	file

- **cout - recap**

- 使用 `cout` 需要 `#include <iostream>`。

可以用 `cout << x`；输出变量值，也可以用 `<<` 连接多个输出。

manipulators（操纵符）可控制输出格式，例如：

1. `boolalpha` / `noboolalpha`：布尔值输出为 `true/false` 或 `1/0`。
2. `setprecision(n)`：设置小数点后保留 n 位。
3. `fixed` / `scientific` / `defaultfloat`：固定小数、科学计数法、自动选择。
4. `setw(n)`：设置输出宽度。

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     // 1. 基本 cout 演示
7     cout << "cout: " << '\n'
8         << "Today's highest temperature is " << 35.0 << '\n'
9         << "Today's highest temperature is "
10        << setprecision(2) << fixed << 35.0 << '\n'
11        << "Today's highest temperature is "
```

```

12         << scientific << 35.0 << '\n';
13
14         // reset format
15         cout << defaultfloat;
16
17         // 2. setw 演示
18         cout << "Table output:" << '\n';
19         cout << "|" << setw(20) << "Name"
20         << "|" << setw(20) << "Tel" << "|\n";
21         cout << "|" << setw(20) << "Daniel Yue"
22         << "|" << setw(20) << "9xxxxxxx" << "|\n";
23     }

```

 image-20250901201238261

- 演示不同格式输出：

1. 普通浮点数输出、定点格式 (fixed)、科学计数法 (scientific)。
2. 使用 `setw` 对齐输出，制作简单的表格。

- `defaultfloat` 会在 `fixed` 和 `scientific` 之间选择较短表示。

- **Note for boolalpha and noboolalpha**

- 默认是 `noboolalpha`，布尔值用 0/1 表示。
- `boolalpha` 让输出变成 `true/false`。
- `boolalpha` 同样适用于 `cin`（读取时识别 `true/false`），以及 `ofstream/ifstream`。

```

1 // 3. boolalpha 演示
2 bool flag = true;
3 cout << "Default noboolalpha: " << flag << '\n'; // 输出 1
4 cout << boolalpha;
5 cout << "With boolalpha: " << flag << '\n'; // 输出 true
6 cout << noboolalpha;
7
8 return 0;

```

Example for boolalpha

- 使用 `istringstream` 从字符串解析布尔值。
- `istringstream`：从字符串读取。
- `ostringstream`：写入字符串。
- 示例：字符串 `"true false"` 被解析成两个布尔变量 `b1=1`，`b2=0`，因为启用了 `boolalpha`。

```

1 bool b1, b2;
2 istringstream is("true false");
3 is >> boolalpha >> b1 >> b2;
4 cout << "\"true false\" parsed as "
5     << b1 << " " << b2 << '\n'; // 输出：1 0

```

- **File input / output**

- 磁盘文件是一种持久化存储（与内存相对）。

- 文件 I/O 的常见用途：

1. 读取已有文件数据，需了解文件结构。
2. 程序运行时保存状态，再次运行时恢复，需要设计合理的文件结构。

- 读写逻辑往往与文件结构紧密相关。

File output: syntax

- 需要 `#include <fstream>`。
- `ofstream stream_name("file")` 类似于 `cout`，但输出目标是文件。
- 语法：

```
1 ofstream out("path/to/file.txt");
2 out << value;
3 out.close();
```

- 不同平台路径示例 (Windows `c:\\test.txt`，Linux/Mac `./test.txt`)。

Example for file output

- 将 "Hello world" 和数字 1-10 输出到文件 `test.txt`。
- 每一行由一个字符串和一个数字组成。
- 程序运行后，文件内容类似：

```
1 // 2.9 File output example
2 ofstream out("test.txt");
3 for (int i = 1; i <= 10; i++) {
4     out << "Hello world!" << i << '\n';
5 }
6 out.close();
```

• File input

- 文件输入需要 `#include <fstream>`。
- 用法：

```
1 ifstream in("file.txt");
2 in >> value;
3 in.close();
```

- 文件路径可以是绝对路径或相对路径。
- 使用方式与 `cin` 类似。

• Useful functions for file input

- 对 `ifstream fin("input.txt")`：
 1. `fin.eof()` → 是否到达文件末尾。
 2. `fin.fail()` → 输入失败（类型不匹配）。
 3. `fin.clear()` → 重置流状态。
 4. `fin.close()` → 关闭文件。
- 系统有最大文件打开数限制，因此用完文件要及时关闭。

• Using a while-loop for file input

- 当文件包含未知数量的同类数据时, 使用 `while` 循环逐个读取:

```
1 while (in >> buffer) { ... }
```

- 或:

```
1 while (!in.eof()) { ... }
```

- 注意:

- `(in >> buffer)` 会在文件结束或出错时返回 `false`。
- `eof()` 仅在读到文件末尾时为真, 但错误不会触发, 所以还要用 `fail()` 检查。

- 读取的值通常追加到一个容器 (如 `vector`) 。

绝对路径和相对路径:

```
1 ifstream f1("data.txt");           // 相对路径: 读取 /home/alice/project/data.txt
2 ifstream f2("input/values.txt");    // 相对路径: 读取 /home/alice/project/input/values.txt
3 ifstream f3("../other.txt");        // 相对路径: 读取 /home/alice/other.txt
4 ifstream f4("/home/alice/project/input/values.txt"); // 绝对路径
```

Example 1

- 创建 `input_numbers.txt` 文件, 内容是空格分隔的浮点数。
- 程序读取这些数并存入 `vector<double>`, 然后逐行输出。

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 using namespace std;
5
6 int main() {
7     ifstream fin("input_numbers.txt");
8     if (!fin) {
9         cerr << "Error opening file!" << endl;
10        return 1;
11    }
12
13    vector<double> values;
14    double buffer;
15
16    // 读取文件数据
17    while (fin >> buffer) {
18        values.push_back(buffer);
19    }
20
21    fin.close();
22
23    // 输出结果
24    for (double v : values) {
25        cout << v << endl;
```

```

26     }
27
28     return 0;
29 }

```

示例输入文件内容：

```

1 63.6757394446919
2 837.7048648372727
3 167.9612790142754
4 997.2665921982092

```

Solution: 从纯数字文件中读取 double。

```

1  /* 2.14 读取纯 double 文件: input_numbers.txt */
2  void test_read_double() {
3      cout << "== test_read_double() ==\n";
4      ifstream in("input_numbers.txt");
5      vector<double> arr;
6      double tmp;
7      while (in >> tmp) {           // 直到失败 (EOF) 为止
8          arr.push_back(tmp);
9      }
10     in.close();
11     for (double v : arr) cout << v << '\n';
12 }

```

Example 2: 当文件里数字与杂质 (字符串) 混在一起时, 直接读取会失败;

```

1 63.6757394446919 837.7048648372727
2 167.9612790142754
3 asdf
4 997.2665921982092

```

Solution: 出现读取失败时, 用 fail()/clear() 清错, 并把“坏数据”读出丢弃后继续。

```

1  /* 2.16 读取“数+杂质”混合文件: input_double_mix.txt
2      思路: 每次提取 double, 如果失败:
3          - clear() 清除失败标志
4          - 用字符串把这一段坏输入读出来丢弃
5          - 继续下一轮
6  */
7  void read_txt_overcome_error() {
8      cout << "== read_txt_overcome_error() ==\n";
9      ifstream in("input_double_mix.txt");
10     vector<double> arr;
11     double tmp;
12     string dummy;
13     while (!in.eof()) {
14         in >> tmp;
15         if (in.fail()) {           // 这次不是数字
16             in.clear();           // 清错
17             in >> dummy;           // 读出这个“坏词”并丢弃

```

```

18         continue; // 继续下一轮
19     }
20     arr.push_back(tmp);
21 }
22 in.close();
23 for (double v : arr) cout << v << '\n';
24 }

```

Example 3: 混合类型（字符串里含空格 + 数字）更稳妥的读写方案——**改变文件结构**：数字行用流提取，字符串行用 `getline`，或给文件增加结构（分段/打标签）避免歧义。

```

1
2  /* 2.17 更稳妥的混合类型读写
3     方案：改变文件结构，分段写入：
4     - 第一段：数字（每行一个，用 operator>> 读取；行尾用 getline 吃掉残余）
5     - 第二段：字符串（每行一个完整字符串，用 getline 读取）
6     文件结构：
7         NUM <count>
8         <double>
9         ...
10        STR <count>
11        <line of string>
12        ...
13    */
14    void write_mixed_structured() {
15        ofstream out("mixed_structured.txt");
16        // 写 3 个数字
17        out << "NUM 3\n";
18        out << 63.6757394446919 << '\n';
19        out << 837.7048648372727 << '\n';
20        out << 167.9612790142754 << '\n';
21        // 写 2 个字符串（内含空格）
22        out << "STR 2\n";
23        out << "Hello World!1\n";
24        out << "Hello World!2\n";
25        out.close();
26    }
27
28    void read_mixed_structured() {
29        cout << "== read_mixed_structured() ==\n";
30        ifstream in("mixed_structured.txt");
31        string tag;
32        while (in >> tag) {
33            if (tag == "NUM") {
34                int n; in >> n;
35                string dummy; getline(in, dummy); // 吃掉本行残余
36                for (int i = 0; i < n; ++i) {
37                    double x; in >> x;
38                    getline(in, dummy); // 吃掉行尾
39                    cout << "[double] " << x << '\n';
40                }
41            } else if (tag == "STR") {
42                int m; in >> m;
43                string dummy; getline(in, dummy);
44                for (int i = 0; i < m; ++i) {

```

```

45         string s; getline(in, s); // 读取整行字符串
46         cout << "[string] " << s << '\n';
47     }
48     } else {
49         // 未知标签, 跳过当前行
50         string dummy; getline(in, dummy);
51     }
52 }
53 in.close();
54 }

```

Solution (按约定读写文本)

- 文本里既有字符串行又有数字行时, 读写必须遵循同一顺序 (FIFO) 与结构。
- 写: 每行写一个元素; 读: 字符串用 `getline`, 数字用 `operator>>`, 再用一次 `getline` 吃掉行尾残余。

```

1 // 2.18 约定式文本读写
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 void fout_with_convention() {
6     ofstream out("test2.txt");
7     for (int i = 0; i < 10; ++i) {
8         out << "Hello world" << i << '\n'; // 一行一个字符串
9         out << i << '\n'; // 下一行一个整数
10    }
11    out.close();
12 }
13
14 void fin_with_convention() {
15     ifstream in("test2.txt");
16     string sin, dummy;
17     int i;
18     for (int k = 0; k < 10; ++k) {
19         getline(in, sin); // 读一整行字符串
20         in >> i; // 读一个整数
21         getline(in, dummy); // 吃掉本行剩余
22         cout << sin << " | " << i << '\n';
23     }
24     in.close();
25 }

```

- Binary format: 克服纯文本限制
 - 文本无法在字符串中安全放入换行等分隔符; 效率也较低。
 - 用二进制可自定义结构、效率高, 但不可读。
 - 需以 `ios::binary` 打开流, 并用 `write/read` 以字节为单位操作。

Setup for binary file

- 任意对象的地址可用 `reinterpret_cast<const char*>/char*>` 转为字节指针。
- 用 `sizeof(T)` 得到对象/标量的字节数。
- 字符串字节数 $\approx$ `str.size()` (英语单字节假设); 容器大小 $\approx$ `vec.size()*sizeof(T)`。

Example 4 (混合类型的二进制协议)

- **写入顺序**：先写“大小(size)”，再写“数据(data)”；读取时**先读 size 再读 data**，循环此过程。
- 这种“size + payload”的协议可复用到：string、vector<T>、多段混合数据。

```
1 // 2.19-2.21 二进制：size + data 协议
2 #include <fstream>
3
4 void write_binary_mixed() {
5     ofstream wf("mix.bin", ios::binary);
6
7     // 写 string：先写 size，再写字符串数据
8     string s = "Hello world 你好"; // 注意：此处含非 ASCII，仅示意
9     size_t n = s.size();           // 这里假设按字节计数（英文 OK）
10    wf.write(reinterpret_cast<const char*>(&n), sizeof(n));
11    wf.write(s.data(), static_cast<std::streamsize>(n));
12
13    // 写 int
14    int x = 42;
15    wf.write(reinterpret_cast<const char*>(&x), sizeof(x));
16
17    // 写 vector<double>
18    vector<double> v{1.1, 2.2, 3.3};
19    size_t m = v.size();
20    wf.write(reinterpret_cast<const char*>(&m), sizeof(m));
21    wf.write(reinterpret_cast<const char*>(v.data()),
22             static_cast<std::streamsize>(m * sizeof(double)));
23
24    wf.close();
25 }
26
27 void read_binary_mixed() {
28     ifstream rf("mix.bin", ios::binary);
29
30     // 读 string
31     size_t n = 0;
32     rf.read(reinterpret_cast<char*>(&n), sizeof(n));
33     string s(n, '\0');
34     rf.read(s.data(), static_cast<std::streamsize>(n));
35
36     // 读 int
37     int x = 0;
38     rf.read(reinterpret_cast<char*>(&x), sizeof(x));
39
40     // 读 vector<double>
41     size_t m = 0;
42     rf.read(reinterpret_cast<char*>(&m), sizeof(m));
43     vector<double> v(m);
44     rf.read(reinterpret_cast<char*>(v.data()),
45            static_cast<std::streamsize>(m * sizeof(double)));
46
47     rf.close();
48
49     cout << "s = " << s << "\n";
50     cout << "x = " << x << "\n";
51     cout << "v = ";
52     for (double t : v) cout << t << " ";
```

```

53     cout << "\n";
54 }

```

- Solution - Write the data in binary format

1. 首先写入一个字符串（长度 12）、一个 `size_t`、以及一个 `vector<double>`。
2. 使用 `reinterpret_cast<char*>` 将任意数据的地址转成 `char*`，以便 `write/read` 使用。
3. 通过 `sizeof(T)` 获取数据占用的字节数，从而精确写入。
4. 写入流程：
 - 写入字符串长度 `size`。
 - 写入字符串本身。
 - 写入 `size=10`。
 - 写入一个循环生成的向量中的元素。

```

1 // 2.22 solution - write
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     ofstream wf("test.dat", ios::out | ios::binary);
7
8     // 写字符串
9     auto str = "Hello World!";
10    size_t size = str.size();
11    wf.write(reinterpret_cast<char*>(&size), sizeof(size_t));
12    wf.write(reinterpret_cast<char*>(&str[0]), size);
13
14    // 写整数 10
15    size = 10;
16    wf.write(reinterpret_cast<char*>(&size), sizeof(size_t));
17    for (int i = 0; i < 10; ++i) {
18        wf.write(reinterpret_cast<char*>(&i), sizeof(i));
19    }
20
21    // 写 vector<double>
22    vector<double> vec(10);
23    int i = 0;
24    for_each(vec.begin(), vec.end(), [&i](auto &x) {
25        x = (i + 1) * (i + 1);
26        ++i;
27    });
28    size = vec.size();
29    wf.write(reinterpret_cast<char*>(&size), sizeof(size_t));
30    for (auto &v : vec) {
31        wf.write(reinterpret_cast<char*>(&v), sizeof(v));
32    }
33
34    wf.close();
35 }

```

- Solution - Read the data in binary format

1. 用 `ifstream` 打开二进制文件。
2. 按写入的顺序严格读出数据：
 - 先读字符串长度 `size`。
 - 再读取字符串。
 - 再读 `size=10`。
 - 再依次读出向量中的元素。
3. 用 `ostream_iterator` 把结果输出到 `cout`。

核心理想：写和读的顺序必须严格一致，否则数据解释会错乱。

```
1 // 2.23 solution - read
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     ifstream rf("test.dat", ios::binary);
7
8     // 读字符串
9     size_t size;
10    rf.read(reinterpret_cast<char*>(&size), sizeof(size_t));
11    string str(size, 'a');
12    rf.read(reinterpret_cast<char*>(&str[0]), size);
13    cout << str << '\n';
14
15    // 读整数 10
16    rf.read(reinterpret_cast<char*>(&size), sizeof(size_t));
17    vector<size_t> vec(size);
18    for (auto &v : vec) {
19        rf.read(reinterpret_cast<char*>(&v), sizeof(int));
20    }
21    copy(vec.begin(), vec.end(), ostream_iterator<int>(cout, ","));
22    cout << '\n';
23
24    // 读 vector<double>
25    rf.read(reinterpret_cast<char*>(&size), sizeof(size_t));
26    vector<double> vec2(size);
27    for (auto &v : vec2) {
28        rf.read(reinterpret_cast<char*>(&v), sizeof(v));
29    }
30    copy(vec2.begin(), vec2.end(), ostream_iterator<double>(cout, ","));
31    cout << '\n';
32
33    rf.close();
34 }
```